

Machine Learning

Lecture 1: What Does It Mean to Learn?

A textbook treatment

Based on the lecture notes of Mounir Ghogho

September 25, 2026

Contents

How to read this book	2
1 What Does It Mean to Learn?	3
1.1 Learning from Experience	3
1.2 Two Routes to Machine Ability	4
1.3 What Counts as Experience?	8
1.4 Supervised Learning: Learning from Examples	10
1.5 Representations	12
1.6 Four Ingredients of a Learning System	15
1.7 The Challenge of Generalization	17
1.8 A Connection to Information Theory	20
Notes on the original lecture notes	22
Chapter takeaways	23

How to read this book

Each concept in this book is presented in the same rhythm.

- 1. Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
- 2. Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
- 3. Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
- 4. Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
- 5. Examples.** From “trivially small” to “genuinely useful.”
- 6. Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, “say it out loud” version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Basic probability (random variables, conditional distributions, expectations), vectors in $\mathbb{R}^d$, and a first course in calculus. The information-theoretic quantities used (entropy, cross-entropy, Kullback–Leibler divergence) are defined and proved from scratch where they appear.

Chapter 1

What Does It Mean to Learn?

1.1 Learning from Experience

What are we trying to get?

We want machines that can *do things we cannot fully describe*: recognise a handwritten digit, flag a fraudulent payment, translate a sentence. This section pins down what “learning” means so that we can later ask, precisely, what a machine needs in order to do it.

Motivation

A child learns to understand language and to recognise faces. A musician becomes fluent through practice. A student becomes able to solve problems that were unfamiliar a month earlier. In each case something changed in the learner, and the change was driven by *experience*: observation, practice, interaction, instruction, feedback. But experience alone is not enough. A stone exposed to a thousand piano lessons learns nothing. The learner also brings a *capacity to change* in response to experience. For humans and animals this capacity comes from a biological system shaped by evolution and development: the brain and nervous system provide structures that support perception, memory and adaptation.

Learning is what happens between “could not do it” and “can do it”, and experience is the fuel.

A first picture, which we will refine throughout the course, is

Capacity to Learn + Experience $\xrightarrow{\text{Learning}}$ Acquired Abilities.

Definition 1.1.1 (Learning, informal). *Learning* is the process through which experience changes a learner and develops what it can do. The capacity to learn is itself shaped by development and previous experience, so the picture above is a starting point rather than a complete description of biological learning.

What kinds of abilities do we learn?

Many acquired abilities involve four broad activities. They overlap, and they are not an exhaustive classification of intelligence, but they give us a vocabulary that also applies to artificial systems.

Recognise	Predict	Reason	Act
What is it?	What may happen?	What follows?	What should I do?

- *Recognition* identifies objects, people, sounds, words and patterns despite variations in how they appear. You recognise your friend from behind, in poor light, wearing a hat.
- *Prediction* uses previous experience and current information to anticipate what may happen: dark clouds suggest rain, a tilting glass suggests a spill.
- *Reasoning* works through a new problem rather than recalling a stored answer: solving an equation you have never seen, planning a route around a road closure.
- *Acting* includes physical and practical skills acquired through repeated interaction: riding a bicycle, pouring water without spilling.

Language draws on several of these at once: recognising words, predicting what comes next, reasoning about meaning, and acting by producing speech.

Example 1.1.2 (One task, four abilities). A radiologist reading a chest X-ray *recognises* anatomical structures and abnormal shadows, *predicts* how a lesion may evolve, *reasons* from symptoms and image to a differential diagnosis, and *acts* by ordering a test or a treatment. A machine-learning system for radiology may take on any subset of these: a classifier that flags suspicious regions addresses recognition; a model of tumour growth addresses prediction; a treatment-recommendation system addresses decision and action. Keeping the four activities separate helps state precisely what such a system is supposed to do.

This raises the question that has driven much of artificial intelligence:

How can we give machines similar abilities?

Summary: Learning from experience

- Learning = capacity to learn + experience → acquired ability.
- Abilities we care about: recognise, predict, reason, act. Language combines them.
- The question of the course: what does a *machine* need in order to acquire abilities from experience?

1.2 Two Routes to Machine Ability

What are we trying to get?

There are two ways to get a machine to do something: tell it *how* (write the rules), or give it *experience* and let it work the rules out. We want to understand when each route is the right one, and why the second route exists at all.

The first idea: encode human knowledge

One approach is to let humans acquire the knowledge and then transfer it to a machine. The idea predates modern computers: Pascal's seventeenth-century *Pascaline* embodied the procedures of arithmetic in a physical mechanism. Pro-

Route 1: humans learn first, then transfer the result to the machine as explicit instructions.

grammable computers later allowed far richer procedures to be expressed as algorithms. The principle is

Human Knowledge $\longrightarrow$ Explicit Rules or Procedures $\longrightarrow$ Computer.

This remains the right approach for many problems. When we know a reliable procedure for sorting numbers, calculating a tax, checking a password, or solving a well-defined numerical problem, we implement it directly. Learning it again from examples would only add uncertainty.

Example 1.2.1 (Programs that should stay programs).

- *Income tax.* The rule is written in law: brackets, rates, deductions. A program applies it exactly. A learned model would approximate the law from past tax returns and be wrong on rare cases. Nobody wants an approximately correct tax bill.
- *Sorting.* Merge sort is proved correct for every input. There is nothing to learn.
- *Password check.* The procedure is “hash the input and compare.” A learned model would have to be right *every* time, which learning never guarantees.

Early artificial intelligence extended the first idea to domains requiring expertise. In an *expert system*, knowledge obtained from specialists was represented as explicit facts and rules. A medical system could contain rules relating symptoms and test results to diagnoses. Systems such as MYCIN showed that carefully constructed rule bases could support sophisticated reasoning in restricted domains. As such systems became more ambitious, extracting, expressing and maintaining the required knowledge became increasingly difficult. This became known as the *knowledge acquisition bottleneck*.

Example 1.2.2 (The knowledge engineer’s year). Imagine building, in the 1980s, an expert system for diagnosing faults in industrial machines, with two hours per week of access to an experienced engineer. After months the system contains hundreds of rules such as

IF motor temperature is high AND vibration is strong THEN
inspect the bearings.

Three kinds of difficulty appear, and it is worth separating them.

- *Obtaining the knowledge.* The engineer “just knows” that a certain rattle means a loose coupling, and cannot say what “that rattle” is in terms a program can test. Two hours a week is far too little to elicit thousands of such judgements.
- *Representing it as rules.* Real judgements are graded (“fairly high temperature”), interacting (“unless the machine has just started”), and full of exceptions. Encoding them produces long chains of rules that contradict each other in corners nobody anticipated.
- *Unfamiliar situations.* When the factory installs a new generation of machines, faults appear that the engineer has never seen. The rules say nothing, and there is no mechanism for the system to improve from the new failures it observes.

A learning-based approach attacks all three: it extracts patterns from logged sensor readings and repair records rather than from interviews, it represents graded and interacting effects numerically, and it can be retrained on the new machines' data. Expert knowledge remains valuable nonetheless: it tells us *which* measurements to record, it can check whether a learned model's behaviour is physically plausible, and it covers rare but catastrophic faults for which no data exist.

Can we always write the rules?

The difficulty is sharpest for abilities that humans perform easily without being able to describe the procedure they use. Consider handwritten-digit recognition. People recognise the same digit across large changes in handwriting, position, orientation, size and stroke thickness. Writing a program that covers these variations through explicit rules is much harder.

The gap: we can do things we cannot describe.

Example 1.2.3 (Trying to write the rules for a “3”). You are given a 20×20 binary image and must decide whether it shows a 3. Some natural rules:

- R1. There are two “bumps” opening to the left: the rightmost black pixel of the top half and the rightmost black pixel of the bottom half both lie in the right third of the image.
- R2. The middle row contains black pixels only in the right half (the “waist” of the 3).
- R3. The number of black pixels lies between 40 and 90.
- R4. No black pixels touch the left border (a 3 has no left vertical stroke, unlike an 8 or a 0).

Now shift the digit four pixels to the right: R1 still holds, R2 may fail if the waist crosses the centre column, R4 holds. Write it narrowly: R1 may fail because the bumps no longer reach the right third. Rotate by 15° : the waist is no longer on the middle row, so R2 fails. Draw it with a thick stroke: R3 fails because the pixel count doubles.

Every fix creates new problems. “Right third” becomes “right third of the bounding box,” which requires computing the bounding box, which is corrupted by a stray dot. “Middle row” becomes “the row of minimum horizontal extent between the two bumps,” which is ambiguous for a flattened 3 and also matches some 5s and 8s. The pixel-count range widens until it excludes nothing. Rules pile up, each with parameters nobody knows how to set, and the interactions between rules become impossible to reason about. At this point, showing the computer a few thousand labelled 3s and non-3s, and letting it find its own regularities, is the sensible move. The thought experiment reveals the essential difference: explicit programming requires us to *know and state* the procedure; learning requires only that we can *recognise* correct behaviour on examples.

Human vision is the broader version of the same point. We know a great deal about the visual system, yet we do not possess a complete explicit description of the computations that allow people to recognise the enormous variety of objects in natural scenes. This illustrates a distinction that the whole field rests on:

Having an ability $\neq$ having an explicit procedure that describes the ability.

The second idea: make the machine learn

When useful experience is available while a complete procedure is hard to specify, another approach becomes attractive. Instead of transferring the *result* of human learning to a computer, we ask whether the machine can acquire the ability from experience itself. The question changes from “what rules should humans write?” to

How can we give a machine the capacity to learn from experience?

The picture of Section 1.1 now applies to machines:

Capacity to Learn + Experience $\xrightarrow{\text{Learning}}$ Acquired Ability.

Designing computational methods that make this possible is the subject of *machine learning*. The capacity to learn is provided by design choices: what the system can represent, what can change through experience, what information guides those changes, and what counts as good performance. Machine learning still involves programming. Human designers construct the learning system and its learning mechanisms; part of the task-specific behaviour is then acquired from experience rather than written as rules.

Technique: Deciding between the two routes

Ask three questions.

1. *Can a competent person write down the complete procedure?* If yes (tax, sorting, physics simulation), program it.
2. *Can we recognise correct behaviour on examples, and do we have (or can we collect) many examples?* If yes and question 1 was no (digits, speech, fraud), learn it.
3. *What is the cost of a mistake?* Learning gives approximately correct behaviour. When “approximately” is unacceptable (safety interlocks, cryptography), keep the explicit program or wrap the learned component in explicit safeguards.

Most real systems mix both: a learned model produces a prediction and an explicit program decides what to do with it under fixed business rules.

Gotcha

- “Machine learning replaces programming” is false. Learning replaces the *task-specific rules*; the learner itself, the data pipeline and the decision logic are still programmed.
- “Expert knowledge is obsolete” is also false. It decides what to measure, sanity-checks the model, and covers cases with no data.

Summary: Two routes

- Route 1: humans learn, then write rules. Right when the procedure is known and exactness matters.
- Route 2: give the machine experience and a capacity to learn. Right when we can recognise correct behaviour but cannot describe the procedure.

- Expert systems hit the knowledge acquisition bottleneck; digit and object recognition show that ability does not imply a describable procedure.

1.3 What Counts as Experience?

What are we trying to get?

Before choosing any algorithm we must know *what information the learner will receive*. Different forms of experience carry different information, and that difference decides which kind of learning is even possible.

Three forms of experience

Experience can reach a machine in three broad forms.

Examples	Observations	Interactions
input + desired output	observed data only	actions + consequences

- *Examples* pair an input with a desired output: an image with the digit it shows, a house description with its sale price. Examples give *direct guidance* about what the output should be.
- *Observations* are data without a supplied answer: images, text, audio, sensor logs. They contain *regularities* that can be exploited without anyone labelling them.
- *Interactions* arise when an agent acts in an environment and observes what follows. The learner partly *shapes its own future experience*, because its actions change what happens next.

Definition 1.3.1 (The three learning settings). The three forms of experience lead to three broad learning settings.

- *Supervised learning*: experience consists of examples with desired outputs.
- *Unsupervised learning*: the learner searches for useful structure in observations without desired outputs. *Self-supervised learning* also learns from observations, but constructs a learning signal from the observations themselves (Section 1.5).
- *Reinforcement learning*: experience comes through interaction; consequences and rewards provide information about the quality of behaviour.

These are broad settings, not mutually exclusive kinds of system. A single system can use more than one.

Example 1.3.2 (Same hospital, four projects). A hospital holds a large collection of chest X-rays; for some patients a diagnosis was recorded.

1. *Predict whether a new X-ray shows pneumonia*. Experience: images paired with diagnoses. Feedback: whether the prediction matches the recorded diagnosis. **Supervised.**
2. *Organise thousands of unlabelled X-rays into groups with similar visual*

characteristics. Experience: images only. Feedback: none from outside; the learner judges structure by internal criteria such as within-group similarity. **Unsupervised**.

3. *Remove part of each X-ray and predict the missing region*. Experience: images only, but each image supplies its own target (the hidden region). Feedback: how well the reconstruction matches the hidden pixels. **Self-supervised**.
4. *Teach a robotic device to position an imaging sensor, rewarding good positioning and penalising unsafe movements*. Experience: the device's own actions and their consequences. Feedback: delayed and evaluative (a reward), not a correct answer. **Reinforcement**.

The paradigms differ not by the data type (all four use X-rays) but by *what information about “doing well” reaches the learner*: a correct answer per example, nothing external, an answer manufactured from the data itself, or a reward earned through action.

Example 1.3.3 (Same records, three questions). A university has ten years of student records: previous grades, attendance, course choices, and eventual outcome.

- *Will each student graduate?* Input: the record at entry; output: graduated or not. Supervised, with a categorical output: *classification*.
- *What will the final graduation grade be?* Same input; output: a number. Supervised: *regression*.
- *Which students have similar academic trajectories?* Input: the records; no desired output at all. Unsupervised.

The same collection of observations supports several learning problems because the *problem* is defined by the question asked, not by the data alone.

Gotcha

The word “unsupervised” does not mean “no objective.” Grouping X-rays still needs a criterion for what a good grouping is; the difference is that no human supplies the desired answer for each example.

Summary: What counts as experience

- Examples (input + output), observations (data only), interactions (actions + consequences).
- They lead to supervised, unsupervised or self-supervised, and reinforcement learning.
- The paradigm is decided by what feedback about “doing well” reaches the learner, not by the data type.

1.4 Supervised Learning: Learning from Examples

What are we trying to get?

Supervised learning is the setting this course develops mathematically in Chapter 2. Here we want a clear picture of its ingredients: what an example is, where the desired outputs come from, how the machine uses them, and what kinds of outputs it may be asked to produce.

Definition 1.4.1 (Supervised example, dataset). A *supervised example* is a pair (x, y) of an *input* x and a *desired output* y . A *training dataset* is a finite collection of such pairs, $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$. The desired output is called the *target*, the *label*, or the *ground truth*, depending on the application. The learner’s goal is to acquire a model that produces useful outputs for *new* inputs.

Where do labels come from?

The desired output must itself come from somewhere, and this has consequences for what can be learned.

- *Measurement.* Temperature, electricity consumption, the recorded sale price of a house. Measurements carry instrument error and recording mistakes.
- *Annotation.* Identifying an object in an image, interpreting a medical scan. Experts disagree, and labels depend on the conventions chosen when the task was defined (is a “dog” label correct for a wolf-dog hybrid?).
- *Delayed outcome.* Whether a customer repays a loan or a machine fails within a month is only known after time has passed. Such labels are available only for old inputs, and the world may have changed since.

“Ground truth” is a strong phrase for something that was measured with error, judged by a person, or observed only later.

The quality of the supervision limits what can be learned. A model trained on labels that are wrong 10% of the time cannot be expected to agree with the (noisy) labels more than about 90% of the time, and its errors will partly mirror the annotators’ errors.

Example 1.4.2 (Phishing detection: designing the experience). Asked to build a phishing-website detector, the first job is to decide what experience to collect, before any algorithm is chosen.

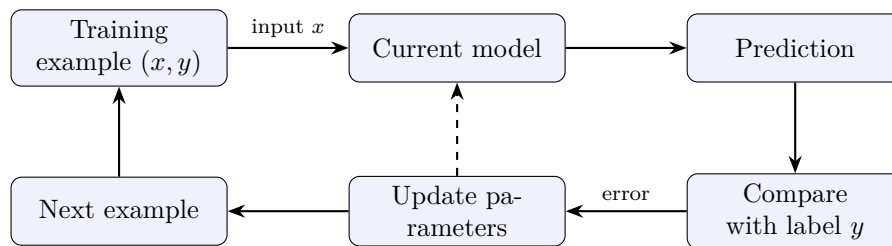
- *Where do examples come from?* Reported phishing pages (from security feeds and user reports) and legitimate pages (from popular sites and from the organisation’s own traffic).
- *How varied should they be?* Many languages, many impersonated brands, pages built with different tools, both crude and sophisticated attacks; otherwise the model learns “phishing = badly spelled English page imitating one bank.”
- *How are correct outcomes established?* A page is phishing if a security analyst confirmed it, or if the impersonated organisation confirmed it. Both are imperfect; some legitimate pages are mislabelled by over-eager reporters.
- *How fast does the world change?* Phishing strategies change monthly. Labels from three years ago describe attacks nobody uses any more, and

today's attacks are absent from the data.

A very large dataset can still give a poor system if the experience is badly chosen: a million pages all from one feed, one language and one year teaches a narrow, outdated notion of phishing. Volume does not substitute for coverage.

How does the machine learn from examples?

During training, the model receives an input and produces a prediction. The prediction is compared with the desired output, and the comparison produces feedback that the learning algorithm uses to change the model. The process continues across the available training experience.



The computational process through which experience changes the model is called *training*. Different learning methods perform the changes differently; Chapter 2 turns this informal cycle into a mathematical problem.

Regression and classification

Definition 1.4.3 (Regression, classification). A supervised problem is a *regression* problem when the desired output is numerical (a real number or a vector of real numbers), and a *classification* problem when the desired output is a category from a finite set of classes. The type of problem is decided by the *output*, not by the input.

Example 1.4.4 (Ten tasks, sorted by their output).

Task	Input	Output	Type
House price	description	price in MAD	regression
Digit recognition	image	one of $\{0, \dots, 9\}$	classification
Spam filter	email	spam / legitimate	classification
Tomorrow's rainfall	weather data	millimetres	regression
Will it rain?	weather data	yes / no	classification
Tumour size	medical image	millimetres	regression
Tumour type	medical image	benign / malignant	classification
Battery lifetime	usage logs	hours	regression
Species	photograph	one of 10 species	classification
Energy demand at 18:00	calendar, weather	megawatts	regression

The same medical image appears twice: predicting its size is regression, predicting its category is classification. The same weather data appears twice as well. The *desired output* decides.

Some supervised problems have richer outputs. *Object detection* returns several objects with their locations (a list of boxes with class labels); *semantic segmentation* assigns a class to every pixel; *machine translation* outputs a sentence.

These are still supervised (examples come with desired outputs), but the output is structured rather than a single number or class.

Prediction, decision, action

Definition 1.4.5 (Prediction, decision, action). A *prediction* expresses what the model says about an unknown output. A *decision* uses the available information together with a criterion to select a response. An *action* is what is carried out when the decision calls for something to be done.

Example 1.4.6 (A screening system). A model predicts that a patient has a 12% probability of a disease (prediction). A protocol says: recommend further testing when the probability exceeds 10% (decision). The test is scheduled and performed (action). Change the protocol threshold to 20% and the same prediction leads to a different decision. In simple applications prediction and decision coincide; they separate as soon as different errors have different consequences, a theme developed precisely in Section 2.2.

Gotcha

- A label is not truth. Ask how it was produced before trusting an accuracy figure.
- “Regression” does not mean “linear regression” and “classification” does not mean “two classes.” The words describe output types.
- A model that predicts probabilities has not yet made a decision. Do not confuse “the model says 0.6” with “the model says yes.”

Summary: Supervised learning

- Examples are (input, desired output) pairs; labels come from measurement, annotation or delayed outcomes, each imperfect.
- Training cycle: predict, compare with the label, update, repeat.
- Regression: numerical output. Classification: categorical output. The output decides the type.
- Prediction, decision and action are distinct steps; they separate when errors have unequal costs.

1.5 Representations: What Is Presented to the Learner?

What are we trying to get?

A learner never sees a house or a photograph; it sees numbers. We want to understand what those numbers should be, why the choice matters enormously, and how modern systems learn the numbers themselves, sometimes without any labels at all.

Feature engineering

For some problems, useful input features can be chosen with ordinary knowledge of the application. A house-price model might receive floor area, location, age, and numbers of bedrooms and bathrooms. Human designers decide these are worth presenting; the model discovers how they relate to price. Some supplied features may turn out to have little influence.

Perceptual problems make the choice much harder. An image contains millions of pixel values; an audio recording is a long waveform; useful information in text depends on relationships among many words. For many years, successful systems relied on representations designed by specialists: edges, corners, textures and shapes for vision; spectral features such as mel-frequency cepstral coefficients for speech. This *feature engineering* let domain knowledge determine what information was presented, and the model learned how to use it. The choice of representation can decide whether learning succeeds at all.

The form of experience says what information exists; the representation says what information the model gets.

Example 1.5.1 (Two representations of the same email). A spam filter could receive (a) the raw sequence of characters, or (b) a vector of counts: how many times each of 10,000 common words appears, whether the sender is in the address book, the number of links, the number of exclamation marks. Representation (b) makes “many links, no known sender, the word *prize*” directly visible as a few coordinates; representation (a) hides it behind thousands of characters whose positions vary from message to message. With a few thousand examples, (b) is learnable and (a) is hopeless. With billions of examples and a model able to build its own representation, (a) becomes viable and can surpass (b), because the designer of (b) forgot features that matter.

Representation learning

Feature engineering raises a further possibility: instead of asking humans to decide all useful features in advance, can the machine discover a useful representation from experience? This is the aim of *representation learning*. The idea is broader than deep learning and predates its success; representations can be learned by many methods.

Definition 1.5.2 (Representation). A *representation* is a transformation of the original input into a form that makes useful structure easier for a learning model to exploit. The two pipelines are

Raw Input $\longrightarrow$ Human-Designed Features $\longrightarrow$ Learned Predictor,
 Raw Input $\longrightarrow$ Learned Representation $\longrightarrow$ Learned Predictor.

Deep learning suits the second pipeline because several layers can construct successive representations, with higher-level features (object parts, shapes) built from lower-level ones (edges, local patterns). The second pipeline does not remove humans from the design: they still choose the architecture, the learning method, the training experience and the objective. What changes is that more of the representation is acquired from data.

Learning representations from observations: self-supervision

Representation learning does not require a label for every observation. In *self-supervised learning* a learning problem is constructed from the observations them-

selves: part of an observation is used to predict another part.

Example 1.5.3 (Three self-supervised tasks).

- *Text.* Hide a word in “The train to _____ leaves at nine” and ask the model to fill it in. The sentence supplies both the context and the answer.
- *Images.* Mask a square region of a photograph and ask the model to reconstruct it from the rest. Success requires knowing what objects and textures look like.
- *Audio.* Remove half a second of speech and predict it from the surrounding sound.

Nobody labelled anything; the data are their own supervision. The task gives the model a reason to discover regularities, and the representation it builds can later be adapted, with a small labelled dataset, to classification, detection or other tasks. Learning from unlabelled observations and transferring the result predates the current generation of large models and remains one of the main uses of representation learning.

From self-supervision to generation

Self-supervised learning can also produce a model whose predictive ability is useful *directly*. Every piece of text already tells us which token followed the previous ones, so a long document provides many training examples for free. An *autoregressive language model* learns to predict the next token from the tokens that came before. Once it can predict, it can *generate*: predict a next token, append it, predict again.

Example 1.5.4 (A next-token model you can compute by hand). Take the tiny corpus

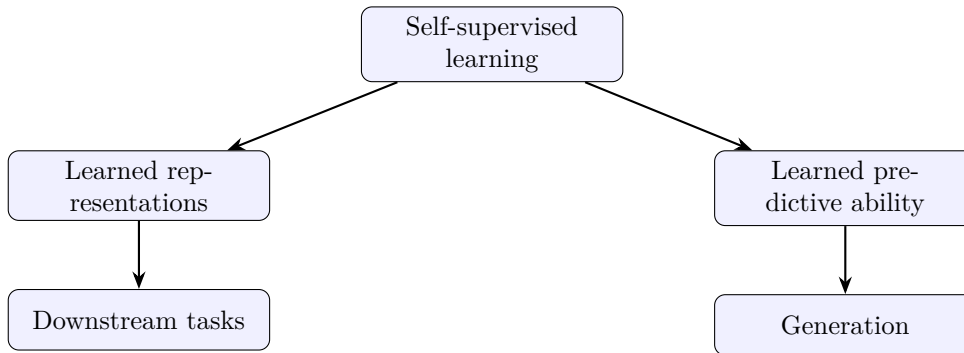
the cat sat on the mat . the cat ate the rat .

The simplest next-token model looks only at the previous word and estimates probabilities by counting. After **the**, the next word is **cat** twice, **mat** once, **rat** once:

$$\mathbb{P}(\text{cat} \mid \text{the}) = \frac{2}{4}, \quad \mathbb{P}(\text{mat} \mid \text{the}) = \frac{1}{4}, \quad \mathbb{P}(\text{rat} \mid \text{the}) = \frac{1}{4}.$$

After **cat**: **sat** and **ate**, each with probability $\frac{1}{2}$. After **sat**: **on** with probability 1. After **on**: **the**. After **mat** or **rat**: the full stop. Nobody wrote a label: the corpus itself says what followed what.

Generation. Start from **the**. Sample the next word from $(\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$, say **cat**; then from $(\frac{1}{2}, \frac{1}{2})$, say **ate**; after **ate** the only continuation is **the**; and so on. One possible output is **the cat ate the mat .**, a sentence that never appears in the corpus. The model has recombined regularities it found. Large language models do the same thing with a context of thousands of tokens and a model with billions of parameters instead of a count table, but the principle, self-supervised next-token prediction followed by repeated sampling, is exactly this.



The two branches are two *uses* of what was learned, not two kinds of model. A generative language model also learns internal representations, and a representation learned by self-supervision can become part of a generative system.

Summary: Representations

- The representation decides which information the model actually receives; a good one can make an impossible problem easy.
- Feature engineering: humans design the features. Representation learning: the machine learns them.
- Self-supervision manufactures targets from the data (fill in the missing word, pixel, sound). Next-token prediction is self-supervised, and repeated sampling from a next-token model is generation.

1.6 Four Ingredients of a Learning System

What are we trying to get?

We want a checklist that applies to *any* learning system, from a count table to a language model, so that we can describe a new application without naming an algorithm and can see what each design choice is responsible for.

Definition 1.6.1 (The four ingredients). Almost any learning system can be examined through four questions, corresponding to four ingredients.

Experience	What information is available?
Model	In what form can acquired knowledge be represented?
Learning algorithm	How does experience change the model?
Evaluation criterion	How is success judged?

This framework begins to answer the question of Section 1.2: what gives a machine the capacity to learn? The model determines what can be represented; the learning algorithm provides the mechanism through which experience changes it; the evaluation criterion indicates which changes are useful; the experience determines what evidence exists in the first place. Representation learning adds one possibility: the way the input is internally represented can itself be shaped by experience.

Example 1.6.2 (Four systems, four ingredients).

	Experience	Model	Learning algorithm	Evaluation
Bigram text model (Ex. 1.5.4)	a corpus	a table of next-word probabilities	count and normalise	probability assigned to held-out text
Digit classifier	labelled images	a function from pixels to 10 probabilities	adjust parameters to reduce error on examples	accuracy on new images
Fraud detector	past transactions with confirmed outcomes	a function from transaction features to fraud probability	same, but with rare positives weighted up	cost of missed fraud plus cost of blocked customers
Warehouse robot	its own actions in simulation	a policy from sensor readings to motor commands	reinforcement from rewards	task completion and safety in the real warehouse

Each row can be discussed, criticised and improved without naming a single algorithm. That is the point of the checklist.

Example 1.6.3 (Designing an application from scratch). *Task: predict, from a farm's sensor data, whether an irrigation pump will fail in the next week.*

1. *Experience.* Two years of hourly readings (current draw, vibration, temperature, flow) from 300 pumps, with the maintenance log recording each failure and its date. Each week of readings becomes an input; the label is whether a failure occurred in the following week.
2. *Model output.* A probability of failure within seven days, so that the maintenance team can rank pumps.
3. *Information for improvement.* Disagreement between predicted probability and what actually happened, aggregated over many pump-weeks. Failures are rare, so the learner must be told that missing one is much worse than a false alarm.
4. *Judging success.* On pumps and months not used during development: how many failures were flagged in advance, and how many unnecessary inspections were triggered, converted into money.

Ingredient by ingredient: (1) is experience, (2) is the model (its output form), (3) is the learning algorithm's signal, (4) is the evaluation criterion.

Technique: Writing the four ingredients for a new problem

- Start with the *evaluation criterion*: what would make the client say "it works"? Everything else must serve it.
- Then *experience*: what is logged already, what can be labelled, what changes over time.
- Then the *model output*: a number, a class, a probability, a ranking, a

sentence?

- Only then the *learning algorithm*. It is the ingredient with the most textbooks and the least influence on whether the project makes sense.

Summary: Four ingredients

- Experience, model, learning algorithm, evaluation criterion.
- The capacity to learn lives in the last three; the evidence lives in the first.
- Describe any application through the four questions before choosing methods.

1.7 The Challenge of Generalization

What are we trying to get?

Experience is always finite, and the situations a deployed system meets are new. We want a precise name for “being useful on new situations,” a clear reason why performance on the training examples does not settle it, and a way to test for it.

Finite experience, infinite situations

A digit recogniser cannot see every handwritten digit that will ever be produced; a medical system cannot observe every future patient; a robot cannot encounter every situation before deployment. A useful learner must acquire something from finite experience that remains useful when a new input appears.

Doing well on what you have seen is not the goal. It is a side effect, and a dangerous one to optimise for.

Definition 1.7.1 (Generalization). *Generalization* is the ability of a learned system to perform well in new situations that were not encountered during learning. The central challenge of machine learning is

$$\text{Finite Experience} \xrightarrow{\text{Learning}} \text{Ability Useful in New Situations.}$$

The following small proposition is the mathematical root of the difficulty. It says that the training data never pin down the rule.

Proposition 1.7.2 (Finite data are consistent with infinitely many rules).

Let $\mathcal{X}$ be an infinite input space, let $\mathcal{Y}$ contain at least two elements, and let $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ be any finite dataset. Then there are infinitely many functions $f : \mathcal{X} \rightarrow \mathcal{Y}$ that agree with the data, i.e. $f(x^{(i)}) = y^{(i)}$ for all i , and any two of them may disagree on every input not in the data.

Proof. Let $S = \{x^{(1)}, \dots, x^{(N)}\}$, a finite set, so $\mathcal{X} \setminus S$ is infinite. Fix two distinct outputs $y_0 \neq y_1$ in $\mathcal{Y}$. For each subset $T \subseteq \mathcal{X} \setminus S$ define $f_T(x) = y^{(i)}$ if $x = x^{(i)} \in S$, $f_T(x) = y_1$ if $x \in T$, and $f_T(x) = y_0$ otherwise. Every f_T agrees with the data. Distinct subsets $T \neq T'$ give distinct functions (they differ on any $x \in T \Delta T'$), and an infinite set has infinitely many subsets. Taking $T = \mathcal{X} \setminus S$ and $T' = \emptyset$ gives two data-consistent functions that disagree on every input outside the data. $\square$

Key insight

Off the training inputs, the data say nothing. Whatever a learned system does there comes from its *model* and its *learning algorithm*, that is, from assumptions built in by the designer. Generalization is therefore never a property of the data alone; it is a property of the match between the assumptions and the world. Chapter 2 introduces the formal objects (hypothesis spaces, risks) needed to study this match.

Example 1.7.3 (The perfect memoriser). The function “look up x in the training set and return its label; otherwise return class 0” has zero error on the training examples. It has learned nothing. A learner evaluated only on its training data cannot distinguish this lookup table from a genuinely useful model. This is why performance on previously encountered examples gives only part of the evidence about what has been learned.

Shortcuts: right answers for the wrong reasons

Example 1.7.4 (Wolves and snow). A system learns to distinguish photographs of wolves from dogs and performs extremely well on the developers’ examples. After deployment its performance drops. Investigation shows that most wolf photographs used in learning contained snow, whereas most dog photographs did not.

1. *Initial success.* “White background $\Rightarrow$ wolf” is a rule consistent with almost all of the training examples (Proposition 1.7.2 guarantees such alternative rules exist). It is far simpler to detect than wolf anatomy, so the learner adopted it.
2. *Failure on new photographs.* Deployed users photograph dogs in snow and wolves in summer. The shortcut now gives systematically wrong answers, because the correlation it exploited was a property of the collection, not of wolves.
3. *Diagnosis.* Two test photographs: a husky standing in snow, and a wolf on green grass. A system that learned about wolves classifies them correctly; a system that learned about snow gets both wrong.

Nothing here assumes anything about the model’s internals. The reasoning uses only the definition of generalization: the system performed well on the situations it had seen and poorly on new ones, and the explanation is a regularity that held in the experience but not in the world.

Example 1.7.5 (Quantity versus diversity). A company must choose one image collection for recognising ten bird species: Dataset A, one million photographs from one camera in one park; or Dataset B, fifty thousand photographs by many people in many countries, seasons, weathers, backgrounds and angles. For a system to be used by the public, B is the better choice: A’s million images share lighting, background, altitude and season, so many of the regularities a learner will find in A (this park’s foliage, this camera’s colour balance) will not hold elsewhere, exactly as in Example 1.7.4. B covers more of the situations the deployed system will meet. A would be preferable if the system is to be deployed *in that park with that camera*, or if the species

are so similar that the sheer number of examples is needed to tell them apart and the deployment conditions match. The lesson: the amount of experience and its *representativeness* of future situations are different quantities, and the second is the one generalization depends on.

Example 1.7.6 (The robot that learned in the wrong world). A warehouse robot trained entirely in a simulator with perfectly flat floors, identical box friction and constant lighting performs well in simulation and poorly in the real warehouse. If only the simulator can be changed, vary the things that vary in reality: floor slope and bumps, box weights and surface friction, lighting, sensor noise, worker behaviour, wheel slippage. A robot exposed to a range of conditions must acquire behaviour that works across them, rather than behaviour tuned to one idealised world. Making the simulator *visually* more realistic would not by itself help: if the floor is still perfectly flat, the robot still learns that wheels never slip. What matters is covering the variations that affect the decisions, not photorealism.

Example 1.7.7 (A claim worth questioning). An advertisement says: “Our AI has learned every possible situation from millions of examples.” The claim confuses situations that were *observed* with situations that *may occur*. Millions of examples are a finite sample; the situations a deployed system will face are not a fixed list and include ones nobody has recorded, and the world changes after the data were collected. What a system can honestly claim is to have learned regularities from the examples it saw that *are expected* to carry over to similar new situations. How similar, and with what guarantee, is exactly the study of generalization, and no amount of data makes the question disappear.

Technique: Testing for generalization without opening the model

- *Hold out.* Evaluate on examples the learner never saw. (Chapter 2 makes this precise with validation and test sets.)
- *Break the shortcuts.* Construct test inputs where suspected spurious cues are decoupled from the label (dog in snow, wolf on grass).
- *Shift the conditions.* Evaluate on data from a different camera, hospital, country or year than the training data. A large drop reveals reliance on collection-specific regularities.
- *Ask what the data cannot know.* List the situations absent from the experience; the model’s behaviour there is an assumption, not a learned fact.

Gotcha

- High accuracy on the training examples is compatible with having learned nothing (the memoriser) or the wrong thing (the snow detector).
- More data helps only if it covers more of the situations that will occur. A billion photographs of one park are one situation.

Summary: Generalization

- Generalization: performing well in situations not encountered during learning.
- Finite data are consistent with infinitely many rules (Proposition 1.7.2); what a learner does off the data comes from its built-in assumptions.
- Shortcuts (snow for wolves) give excellent training performance and fail on new data. Test on held-out, shifted and shortcut-breaking inputs.

1.8 A Connection to Information Theory**What are we trying to get?**

We want a numerical way to say how good a set of predicted probabilities is, and a way to talk about how much information a representation keeps. Both come from information theory, and the single inequality proved here is the reason cross-entropy is the workhorse loss of classification in Chapter 2.

Entropy, cross-entropy, divergence

Recall that *entropy* describes uncertainty, and *information* tells us how much uncertainty is removed by observing something. From a learning perspective, an input is useful for prediction when it contains information about the output.

Entropy: how uncertain the truth is. Cross-entropy: how surprised you are, on average, when you believed q and the truth was drawn from p .

Definition 1.8.1 (Entropy, cross-entropy, Kullback–Leibler divergence). Let $p = (p_1, \dots, p_C)$ and $q = (q_1, \dots, q_C)$ be probability distributions on the same finite set of C outcomes, with $q_c > 0$ whenever $p_c > 0$. Define

$$H(p) = -\sum_c p_c \log p_c, \quad H(p, q) = -\sum_c p_c \log q_c,$$

$$D_{\text{KL}}(p \parallel q) = \sum_c p_c \log \frac{p_c}{q_c},$$

with the convention $0 \log 0 = 0$. Base-2 logarithms measure in bits; natural logarithms, customary in machine learning, measure in nats.

Expanding the divergence, $D_{\text{KL}}(p \parallel q) = \sum_c p_c \log p_c - \sum_c p_c \log q_c = -H(p) + H(p, q)$, so

$$H(p, q) = H(p) + D_{\text{KL}}(p \parallel q). \quad (1.1)$$

Theorem 1.8.2 (Gibbs' inequality). $D_{\text{KL}}(p \parallel q) \geq 0$, with equality if and only if $p = q$. Equivalently, $H(p, q) \geq H(p)$ with equality iff $q = p$: among all distributions q , the expected surprise $H(p, q)$ under truth p is smallest when q is the truth.

Proof. Step 1 (a one-line inequality about logarithms). For every $t > 0$, $\ln t \leq t - 1$, with equality iff $t = 1$. Indeed $g(t) = t - 1 - \ln t$ has $g'(t) = 1 - 1/t$, negative on $(0, 1)$ and positive on $(1, \infty)$, so g has its unique minimum at $t = 1$ where $g(1) = 0$.

Step 2 (apply it term by term). Let $S = \{c : p_c > 0\}$. Using natural

logarithms,

$$-D_{\text{KL}}(p\|q) = \sum_{c \in S} p_c \ln \frac{q_c}{p_c} \leq \sum_{c \in S} p_c \left(\frac{q_c}{p_c} - 1 \right) = \sum_{c \in S} q_c - \sum_{c \in S} p_c = \sum_{c \in S} q_c - 1 \leq 0,$$

since $\sum_{c \in S} q_c \leq 1$. Hence $D_{\text{KL}}(p\|q) \geq 0$. Changing the base of the logarithm multiplies everything by a positive constant, so the sign is unaffected.

Step 3 (equality). Equality requires equality in Step 1 for every $c \in S$, i.e. $q_c = p_c$ for all c with $p_c > 0$, and also $\sum_{c \in S} q_c = 1$, which forces $q_c = 0$ whenever $p_c = 0$. So $q = p$. Conversely $p = q$ gives every term $\log 1 = 0$. $\square$

Key insight

The whole theorem is the tangent-line inequality $\ln t \leq t - 1$, applied to the ratios q_c/p_c and averaged under p . The averaging turns a family of pointwise inequalities into one statement about the whole distribution. This trick, “bound the log by its tangent, then average,” recurs throughout machine learning.

Example 1.8.3 (Numbers). Let the truth be $p = (0.7, 0.3)$ over two classes. In bits, $H(p) = -0.7 \log_2 0.7 - 0.3 \log_2 0.3 \approx 0.881$.

Prediction q	$H(p, q)$	$D_{\text{KL}}(p\ q)$	Comment
(0.7, 0.3)	0.881	0	the truth: minimum possible
(0.5, 0.5)	1.000	0.119	hedging costs 0.119 bits
(0.9, 0.1)	1.103	0.222	overconfident in the right direction: worse than hedging
(0.3, 0.7)	1.370	0.489	wrong direction
(0.99, 0.01)	2.003	1.122	very overconfident: the 0.3 mass pays $\log_2 100$ each

Two lessons. The floor 0.881 is a property of the *problem*: no prediction beats it, because the outcome is genuinely uncertain. And overconfidence is punished sharply: (0.9, 0.1) is further from the truth in cross-entropy than the uninformative (0.5, 0.5), even though it favours the right class.

Why this matters for learning

Two connections deserve to be kept in mind for the rest of the course.

- *Cross-entropy as a criterion.* A classifier may assign probabilities to several classes. Cross-entropy compares those probabilities with what actually occurs: giving high probability to the correct class incurs a small penalty, giving it little probability a large one. By Theorem 1.8.2, the expected penalty is minimised exactly when the predicted probabilities equal the true conditional probabilities. A familiar information-theoretic quantity thus becomes one of the most widely used learning criteria (Section 2.5).
- *Representations and information.* Raw observations contain many kinds of information, only some of it useful for a given purpose. Representation learning asks how data can be transformed so that useful structure becomes easier to exploit. Information theory supplies the vocabulary: an informative representation retains what is relevant about the output and discards the rest.

Technique: Computing with entropies

- $H(p, q) = H(p) + D_{\text{KL}}(p||q)$: compute the entropy once, then only the divergence changes with the prediction.
- Cross-entropy of a *one-hot* truth ($p_y = 1$) is simply $-\log q_y$: the penalty for the probability given to the observed class. This is the form used for training in Chapter 2.
- D_{KL} is not symmetric: $D_{\text{KL}}(p||q) \neq D_{\text{KL}}(q||p)$ in general. Check the order before quoting it.
- $q_c = 0$ for an outcome with $p_c > 0$ gives $H(p, q) = +\infty$: never predict probability exactly zero for something that can happen.

Classic problems

- Which q minimises $H(p, q)$? $q = p$, by Gibbs' inequality; the minimum value is $H(p)$.
- p uniform on C outcomes: $H(p) = \log C$. It is the maximum entropy on C outcomes (apply Gibbs with q uniform: $\log C - H(p) = D_{\text{KL}}(p||\text{unif}) \geq 0$).
- Truth $(0.5, 0.5)$, prediction $(0.99, 0.01)$: cross-entropy in bits? $-0.5 \log_2 0.99 - 0.5 \log_2 0.01 \approx 3.33$, versus 1 bit for the honest prediction.

Summary: Information theory

- $H(p, q) = H(p) + D_{\text{KL}}(p||q)$ and $D_{\text{KL}} \geq 0$ with equality iff $q = p$ (Gibbs, Theorem 1.8.2).
- Cross-entropy therefore rewards *calibrated* probabilities, punishes over-confidence, and has an irreducible floor $H(p)$ set by the problem.
- Representation learning can be phrased as keeping the information about the output and discarding the rest.

Notes on the original lecture notes

The Lecture 1 notes are conceptual, and every claim in them was checked against the standard history and vocabulary of the field. The following points are worth a reader's attention.

- **A stray citation marker (Section 14 of the notes).** The sentence ending "remains an important use of representation learning" is followed by the text `:contentReference[oaicite:4]index=4`. This is an artefact of the AI-assisted editing acknowledged at the end of the notes, not part of the content; ignore it.
- **"Ground truth" (Section 8).** The notes rightly warn that the term should be used with care. This chapter goes one step further and treats label quality as an explicit limit on achievable accuracy.
- **Four abilities (Section 2).** The notes state that recognise, predict, reason, act are "not an exhaustive classification." Do not use the list as a taxonomy

in an exam answer; use it as vocabulary.

- **Generalization (Section 17).** The notes define generalization and defer the reasons “why several relationships can explain the same finite observations” to Lecture 3. Proposition 1.7.2 supplies the elementary version of that fact now, since it is what makes the wolves-and-snow story possible.
- **Information theory appendix.** The notes recall entropy and cross-entropy from the information theory course without proof. Theorem 1.8.2 proves the one fact everything rests on.

Chapter takeaways

Summary: What it means to learn

- **Learning** turns experience into ability. Machines can be given abilities by writing rules or by giving them experience plus a capacity to learn; the second route exists because having an ability does not imply having a describable procedure.
- **Experience** comes as examples, observations or interactions, leading to supervised, unsupervised/self-supervised and reinforcement learning. The paradigm is decided by the feedback the learner receives.
- **Supervised learning** pairs inputs with desired outputs, whose quality limits what can be learned; regression and classification are distinguished by the output; prediction, decision and action are distinct.
- **Representations** decide what the model sees. They can be designed or learned, and self-supervision lets them be learned from unlabelled data; next-token prediction turns this into generation.
- **Four ingredients** describe any learning system: experience, model, learning algorithm, evaluation criterion.
- **Generalization** is the real goal. Finite data never determine the rule; training performance is not evidence of it; shortcuts are the typical failure.
- **Cross-entropy** is minimised by the true probabilities (Gibbs’ inequality), which is why it will be our training criterion for classification.